Project: STM32 Tic-Tac-Toe Server with a PC GUI Client

Juwhin Goliath
GLTJUW001

Arseama Gebremichael
GBRARS001

***Abstract*—This report describes the design, implementation and validation of a client–server turn-based game system in which an STM32F051 microcontroller acts as the authoritative game server and a thin PC-side graphical client renders state and forwards player input. The server owns all game state, validates every move, detects terminal outcomes and rejects malformed or illegal requests without corrupting its internal state. The system communicates over a UART link using a newline-terminated ASCII protocol with explicit negative acknowledgements, and is verified by an automated Python test harness that drives the serial interface without human interaction.**

***Index Terms*—STM32, UART, client–server, finite state machine, protocol design, automated testing**

I. Introduction

A. Context

This project implements a small client–server game over a serial link, as set by the EEE3095S Embedded Systems II course project. An STM32F051 microcontroller is the referee and single source of truth: it stores the board, turn and game status, validates every request and returns the authoritative result. The PC client only renders confirmed state and forwards player input; it never decides whether a move is legal.

A client-untrusted architecture was chosen because it places all correctness-critical behaviour in the embedded software, where it can be specified and tested independently of any particular client. Illegal, out-of-turn, out-of-state and malformed requests are rejected with specific NACK codes and leave the server state unchanged, so a faulty client cannot corrupt the game.

The project exercises three areas of embedded practice: interrupt-driven UART reception with safe framing (maximum length, timeout and recovery from garbage); a three-state server finite state machine with a fixed move-validation order; and repeatable validation through an automated Python harness together with host-side C tests that run the same server code on a PC. Beyond the compulsory requirements, the server also provides a computer opponent and a move-history command.

B. Selected Game and GUI Library

Tic-Tac-Toe was chosen because its rules are small enough to be specified completely while it still needs turn control, three kinds of move rejection and two terminal outcomes. The GUI uses Tkinter [2] (Tk 9.0, Python 3.15), ships with the standard Windows and Mac OS Python installers, needs no extra dependency, and the game is a simple grid of buttons and labels. The serial link uses pyserial version 3.5 [4] and the tests use pytest version 9.1.1 [3] . The deliberate departures from the reference protocol are justified in Section II-C.

C. Task Allocation

Table I summarises the division of work. Each member wrote the report sections for their own half, and both reviewed the whole report.

TABLE I
Task allocation

Member	Responsibilities
Arseama GBRARS001 (embedded server, C)	Protocol specification and protocol tests; UART setup, interrupt-driven receive and frame parser (maximum length, timeout, garbage handling); game module (board, validation, turn control, win/draw detection); server FSM (IDLE, IN_PROGRESS, GAME_OVER) with button and LED behaviour; computer opponent and move history (server side); circuit and FSM diagrams.
Juwhin GLTJUW001 (PC side, Python)	Shared serial wrapper (open port, send line, read line with timeout); pytest harness covering every case in the brief; GUI with error display and moves disabled after game over; computer-opponent mode selector and move-history list; GUI flowchart; integration testing on the real board; demonstration video and GitHub repository.

Table II gives the project plan as ordered milestones, each led by one member.

TABLE II
Project plan and milestones

Milestone	Activities	Lead
1. Specification	Choose the game; agree framing, checksum and error codes; write the protocol specification	Arseama
2. PC interface	Serial wrapper, Python fake server, pytest harness	Juwhin
3. Firmware core	UART receive/transmit, frame parser, game module, server FSM, LCD, LED and button	Arseama
4. Integration	Run the harness against the real board and fix mismatches	Both
5. GUI	Tkinter client, GUI flowcharts	Juwhin
6. Extensions	Computer opponent and move history, on the server and in the GUI and tests	Both
7. Validation and handover	Full test runs, screenshots, demo video, report, repository	Both

D. Report Structure

Section II defines the message protocol, covering framing, board encoding, the command set, the error responses and the departures from the reference brief. Section III presents the design diagrams: the server state machine, the GUI flowchart and the circuit diagram. Section IV explains the key code excerpts. Section V describes game logic and server authority, and Section VI covers protocol robustness and integrity. Section VII describes the move history and single-player extras, Section VIII reports the automated tests and the working system, and Section IX concludes. The demonstration video link, the repository link and the references follow.

II. Requirements and Message Protocol

A. Framing and Encoding

Frames are sent over UART at 115200 baud, 8N1, as case-sensitive ASCII with uppercase commands, terminated by `\n` (`\r\n` also accepted). Maximum frame length is 32 bytes excluding the terminator; longer frames yield `NACK:OVERFLOW`, and a partial frame with a gap over 500 ms yields `NACK:TIMEOUT`. An optional suffix `*HH` carries the XOR of all preceding bytes as two hex digits; if present it must be valid (`NACK:CHECKSUM` otherwise), and replies carry no checksum.

The board is nine characters, indexed 0–8 row-major (0 = top-left), using `.` for empty, `X` or `O` for marks, with `X` moving first; for other games the same scheme generalises to N row-major cells with a documented mark alphabet.

B. Command Set

`NEW` is human-vs-human; `NEW:AI` has the server play `O`, appending `:AI:<c>` to each non-terminal reply. Move errors, in order: `CHECKSUM`/`TIMEOUT`/ `OVERFLOW`, `BADCMD`, `NOGAME`, `NOTURN`, `RANGE`, `OCCUPIED`. Button press forces IDLE and emits `EVT:RESET`. Fig. 1 shows a typical session: connect, new game, a legal move, a rejected move, a winning move, a move after game over, and disconnect. The replies shown are those produced by the real server code. Fig. 2 extends this with error responses, the button reset and the computer opponent.

C. Error Responses and Justified Departures

Each `NACK` corresponds to one condition, checked in the order listed: `CHECKSUM` (bad `*HH` suffix), `OVERFLOW` (frame over 32 bytes), `TIMEOUT` (inter-byte gap over 500 ms), `BADCMD` (unknown command or malformed argument), `NOGAME` (command requires an active game), `NOTURN` (mark does not match the current turn), `RANGE` (cell outside 0–8), and `OCCUPIED` (cell already taken). A busy `CONNECT` or `NEW` returns `NACK:BUSY`.

Four departures from the reference brief are deliberate. First, `WIN:X:<board>` and `DRAW:<board>` carry the final board so the GUI can redraw from the reply alone. Second, the optional mark in `MOVE:<M>:<i>` exposes `NACK:NOTURN` and keeps the client untrusted, while bare `MOVE:<i>` still works. Third, `STATE` reports turn and status so the GUI can

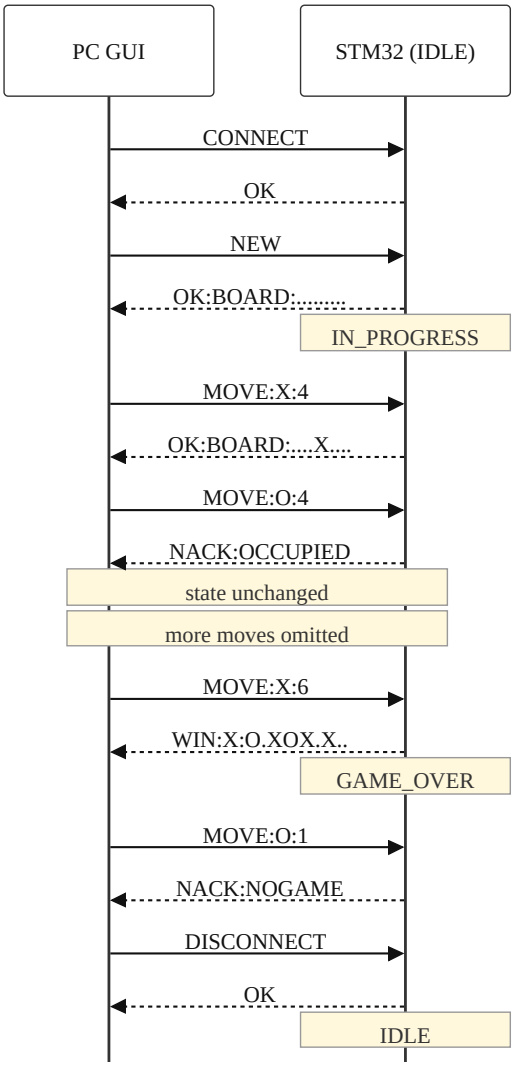


Fig. 1. Message sequence for a game ending in a win.

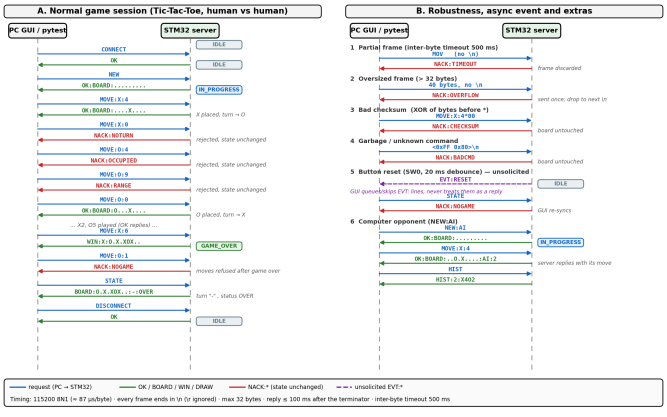


Fig. 2. Message sequence with error responses, justified departures and player vs AI

Request	State	Response
CONNECT	IDLE	OK
CONNECT	other	NACK:BUSY
NEW, NEW:AI	IDLE, GAME_OVER	OK:BOARD:<9>
NEW, NEW:AI	IN_PROGRESS	NACK:BUSY
MOVE:<i>, MOVE:<M>:<i>	IN_PROGRESS	OK:BOARD:<9> | WIN:<M>:<b> | DRAW:<b>
MOVE:...	IDLE, GAME_OVER	NACK:NOGAME
STATE	IN_PROGRESS, GAME_OVER	BOARD:<9>:<turn>:<status>
STATE	IDLE	NACK:NOGAME
RESIGN	IN_PROGRESS	OK:GAMEOVER
RESIGN	other	NACK:NOGAME
DISCONNECT	any	OK
other	any	NACK:BADCMD

TABLE III
COMMAND/RESPONSE TABLE.

resync after a reconnect or button reset. Finally, the added errors BUSY, TIMEOUT, OVERFLOW, and CHECKSUM give specificity beyond a single BADCMD, and DISCONNECT is accepted in any state so a crashed GUI cannot leave the server stuck.

III. Specification and Design

A. Embedded Server Finite State Machine

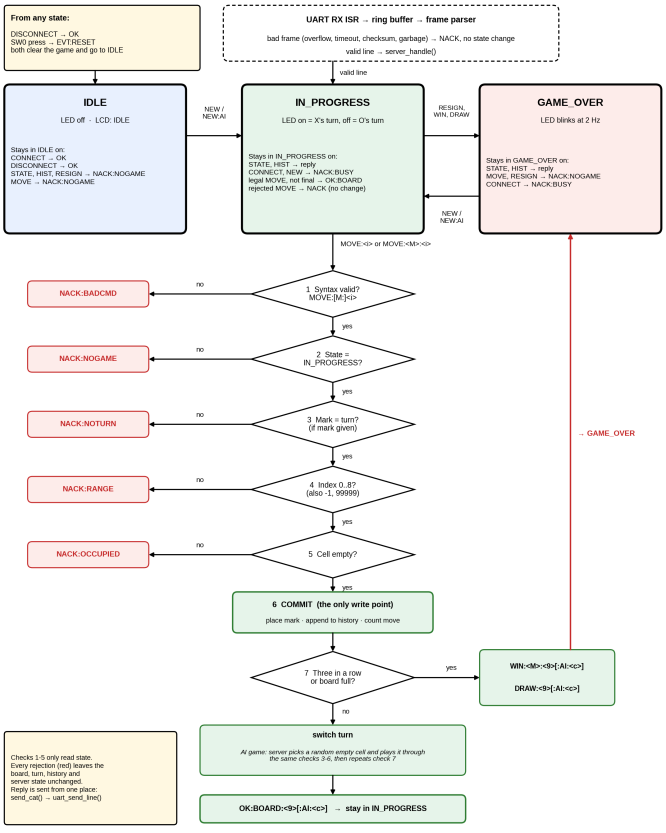


Fig. 3. Embedded server finite state machine.

1) Server FSM: The server is a three-state machine (IDLE, IN_PROGRESS, GAME_OVER). Table IV lists every transition. "–" means the state is unchanged; responses prefixed NACK: leave all game and server state untouched.

2) Game FSM: The pure game logic is a nested two-terminal machine reached only while the server is in PLAY. Table V shows its transitions; all rejected moves return G_* codes and, by construction, do not mutate any state.

B. GUI Application Flowchart

The GUI is a thin client built around one event loop. Fig. 4 shows connection, new game (human or computer opponent), resign, the 150 ms poll for unsolicited lines such as EVT:RESET, and disconnect. Fig. 5 expands move handling: the GUI sends the request without checking legality, and the board is redrawn only from a valid server reply. Timeouts, malformed replies and server rejections each end in a specific message with the board unchanged.

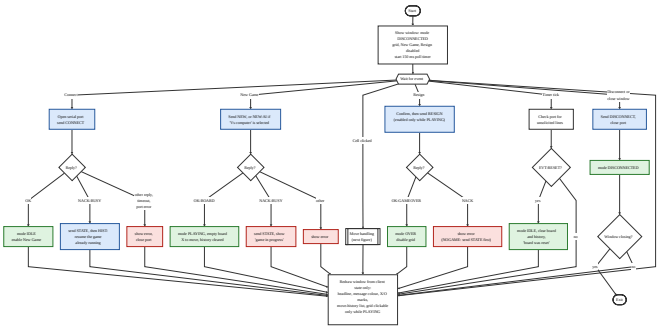


Fig. 4. PC GUI application flowchart.

C. Circuit Diagram

The STM32 connects to the PC over USART1 (PA9/PA10, 115200 baud, 8N1) through the board's built-in ST-LINK USB virtual COM port, while the SW0 button (PA0, active low) resets the game to IDLE and the green LED (PB11) shows the turn (on for X, off for O) and blinks at 2 Hz when the game is over.

IV. Implementation

The move handler is the centre of the server's authority. Every request reaches game_move() only after the parser has accepted the frame and server_handle() has confirmed the command syntax and the PLAY state; the function

TABLE IV
SERVER FSM TRANSITIONS. $M \in \{X, O\}$, $i \in 0..8$, c = AI CELL.

From	Event	To	Response
IDLE	`CONNECT`	IDLE	`OK`
IDLE	`CONNECT` (already connected)	–	`NACK:BUSY`
IDLE	`NEW` / `NEW:AI`	PLAY	`OK:BOARD:<9>`
IDLE	`MOVE`, `STATE`, `HIST`, `RESIGN`	–	`NACK:NOGAME`
PLAY	`NEW` / `NEW:AI`	PLAY	`NACK:BUSY`
PLAY	`MOVE` legal, game continues	PLAY	`OK:BOARD:<9>[:AI:<c>]`
PLAY	`MOVE` legal, win	OVER	`WIN:<M>:<9>[:AI:<c>]`
PLAY	`MOVE` legal, draw	OVER	`DRAW:<9>[:AI:<c>]`
PLAY	`MOVE` wrong turn	–	`NACK:NOTURN`
PLAY	`MOVE` out of range	–	`NACK:RANGE`
PLAY	`MOVE` occupied	–	`NACK:OCCUPIED`
PLAY	`MOVE` malformed	–	`NACK:BADCMD`
PLAY	`RESIGN`	OVER	`OK:GAMEOVER`
PLAY	`STATE` / `HIST`	–	`BOARD:...` / `HIST:...`
OVER	`MOVE`	OVER	`NACK:NOGAME`
OVER	`RESIGN`	OVER	`NACK:NOGAME`
OVER	`NEW` / `NEW:AI`	PLAY	`OK:BOARD:<9>`
OVER	`STATE` / `HIST`	–	`BOARD:...` / `HIST:...`
any	`DISCONNECT`	IDLE	`OK`
any	button press (debounced)	IDLE	`EVT:RESET`
any	unknown / malformed frame	–	`NACK:BADCMD`

TABLE V
GAME FSM TRANSITIONS (`GAME_MOVE()` RETURN CODES).

From	Event	To	Return
GS_PLAY	legal move, no win, <9 moves	GS_PLAY (turn flips)	`G_OK`
GS_PLAY	legal move completes a line	GS_WIN (winner = turn)	`G_OK`
GS_PLAY	legal move, 9th cell, no line	GS_DRAW	`G_OK`
GS_PLAY	`mark` given and $\neq$ turn	GS_PLAY	`G_NOTURN`
GS_PLAY	$i < 0$ or $i > 8$	GS_PLAY	`G_RANGE`
GS_PLAY	cell already occupied	GS_PLAY	`G_OCCUPIED`
GS_WIN	any `MOVE`	GS_WIN	`G_OVER`
GS_DRAW	any `MOVE`	GS_DRAW	`G_OVER`
any	`game_reset()`	GS_PLAY (empty board, turn X)	–

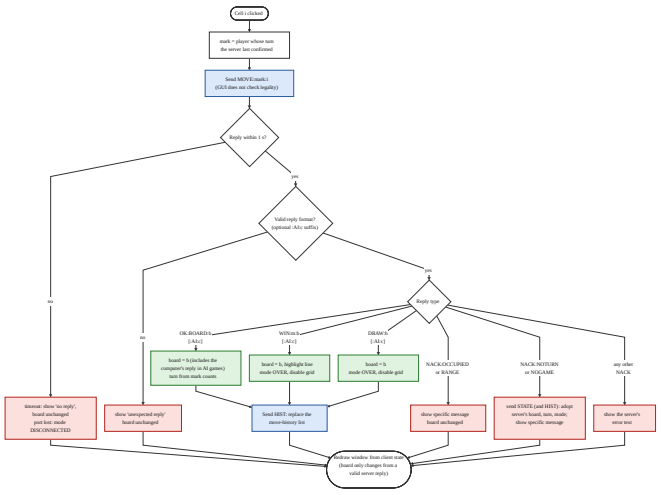


Fig. 5. Move handling (expands the move block of Fig. 4).

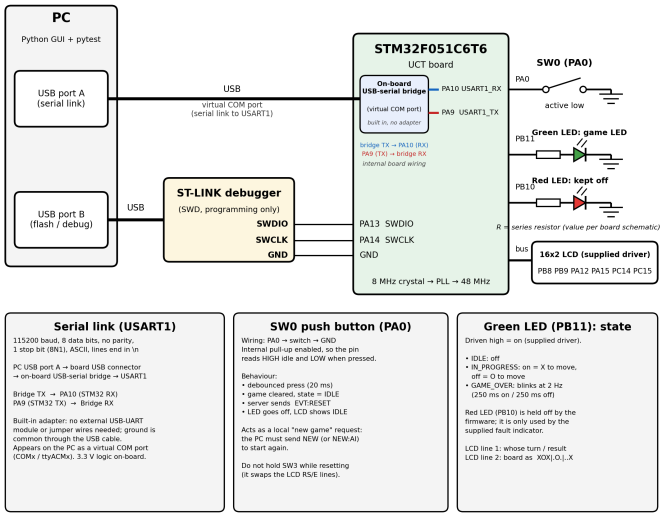


Fig. 6. Circuit diagram: UART link, button and LED.

then performs the remaining rule checks in a fixed order and writes state only after all of them pass (Listing 1).

Listing 1. Validated move commit in `game_move()` (abridged).

```
move_result_t game_move(char mark, int idx)
{
    if (status != GS_PLAY)          return G_OVER;
    if (mark != 0 && mark != turn) return G_NOTURN;
    if (idx < 0 || idx > 8)         return G_RANGE;
```

```
    if (board[idx] != '.')          return G_OCCUPIED;

    /* commit point: writes below this line only */
    board[idx] = turn;
    hist[moves*2u]     = turn;
    hist[moves*2u + 1u] = (char)('0' + idx);
    hist[moves*2u + 2u] = '\0';
    ++moves;

    for (uint8_t l = 0; l < 8u; ++l)
        if (board[lines[l][0]] == turn &&
            board[lines[l][1]] == turn &&
            board[lines[l][2]] == turn) {
            status = GS_WIN; winner = turn;
            return G_OK;
        }
    if (moves == 9u) { status = GS_DRAW; return G_OK; }

    turn = (turn == 'X') ? 'O' : 'X';
    return G_OK;
}
```

Three implementation decisions are non-obvious. Reception is interrupt-driven: `USART1_IRQHandler` calls `uart_irq_handler()`, which pushes each byte into a 128-byte power-of-two ring buffer with a head index written only by the ISR and a tail index written only by the main loop; `uart_rx_pop()` drains it, and a full buffer drops bytes instead of blocking, so the server never stalls on the link. The parser keeps a 32-byte counter plus `discarding` and `last_byte_ms` flags: the 33rd byte triggers one `NACK:OVERFLOW` and a discard state that ignores bytes until the next newline, while `parser_poll_timeout()` reports `NACK:TIMEOUT` and clears a stalled partial frame after 500 ms, resynchronising silently if it was already discarding. Finally, `game_move()` places every check above a single marked commit point, so a rejected request returns a `G_*` code having written nothing; this makes the "rejected requests cannot change state" guarantee structural rather than a matter of care.

The GUI is single-threaded. A request blocks for at most 1 s waiting for one reply, which is acceptable because the server answers in milliseconds. Unsolicited lines such as `EVT:RESET` cannot be waited for this way, so a 150 ms Tk timer calls `poll()`, which reads only what is already buffered (`in_waiting`) and never blocks (Listing 2). Any `EVT:` line seen while waiting for a reply is queued instead of being mistaken for that reply.

Listing 2. GUI poll for unsolicited lines.

```
def poll(self):
    if self.link is None:
        return False
    try:
        events = self.link.poll_events()
    except (LinkError, OSError) as exc:
        self._lost(str(exc) or exc.__class__.
    __name__)
        return True
    for event in events:
        self._on_event(event)
    return bool(events)
```

V. Game Logic and Server Authority

- **Server authority:** the STM32 is the sole authority. All moves pass through `server_handle()` and `game_move()`.
- **Validation order:** frame/checksum → command syntax → server state `PLAY` → mark matches turn → index in 0–8 → cell empty. Only then is the move committed.
- **Turn control:** X always starts; bare `MOVE:<i>` uses the current turn, while `MOVE:<M>:<i>` must match it, else `NACK:NOTURN`. In `NEW:AI`, human is X and the server plays O; `MOVE:O:<i>` is rejected.
- **Win/draw detection:** after a legal commit, the 8 winning lines are checked; a win sets `GS_WIN` and records the winner without swapping turn. A full board with no line sets `GS_DRAW` at 9 moves.
- **No mutation on rejection:** all checks precede the commit point in `game_move()`. Rejected requests leave board, turn, move count, history, winner, and status unchanged; rejected moves are never recorded in `HIST`.
- **AI integrity:** the computer move is chosen only after a legal human move, then played through the same `game_move()` validation; the reply board includes the AI move and appends `:AI:<cell>` only when the game did not already end. Each rejection test in Section VIII re-reads the board with STATE and asserts it is unchanged.

VI. Protocol Robustness and Integrity

- **Framing:** ASCII commands terminated by `\n`; `\r` is ignored. Frames are limited to 32 bytes excluding the terminator, and empty frames are rejected with `NACK:BADCMD`.
- **Specific error reporting:** malformed syntax or non-printable bytes yield `NACK:BADCMD`; oversized frames yield `NACK:OVERFLOW`; stalled partial frames yield `NACK:TIMEOUT`; invalid optional checksums yield `NACK:CHECKSUM`. Game-level failures are distinguished as `NOGAME`, `NOTURN`, `RANGE`, `OCCUPIED`, and `BUSY`.
- **Overflow handling:** once the 32-byte limit is exceeded, the parser sends `NACK:OVERFLOW` once, enters a discard state, and ignores bytes until the next `\n`, then resynchronises.
- **Timeout recovery:** if no byte arrives for 500 ms, an incomplete frame is discarded and `NACK:TIMEOUT` is sent. If already discarding after overflow, the parser resynchronises silently at timeout.
- **Integrity checking:** an optional `*HH` suffix carries an XOR checksum over all bytes before `*`. If present it must be valid; otherwise the frame is rejected with `NACK:CHECKSUM`. No CRC is used.
- **UART error containment:** overrun, framing, and noise errors clear the USART flags and inject a non-printable `0xFF` marker, forcing the parser to reject the damaged frame rather than silently accepting a shortened command.
- **Non-blocking reception:** the interrupt-driven RX ring buffer drops bytes on overflow instead of blocking; TX waits are bounded, so a faulty link cannot stall the main loop.

- **State-safe validation:** command syntax and server state are checked before dispatch, and `game_move()` validates turn, range, and occupancy before any board mutation. Rejected moves therefore leave game, history, and turn state unchanged.
- **Graceful recovery:** `DISCONNECT` is accepted in any state and returns the server to `IDLE`; the debounced button reset also forces `IDLE` and emits asynchronous `EVT:RESET`. `NEW`/`NEW:AI` recover from `IDLE` or `GAME_OVER`.

VII. Extra Features

Two optional stretch features were added after the compulsory system was complete: a move history and a single-player mode against a computer opponent. Both live on the server, so the client remains untrusted.

A. Move History

The game module records every accepted move as a mark and a cell index, for example `X4O0X8`. The record is a fixed 19-byte string (nine moves of two characters plus a terminator) written only at the commit point of `game_move()`, so a rejected move can never appear in it. The new `HIST` command returns `HIST:<n>:<moves>`, where `<n>` is the move count; in `IDLE` it returns `NACK:NOGAME`. The history is kept after the game ends and cleared by `NEW`, `DISCONNECT` and the button reset. The GUI requests it after every accepted reply and displays the list, so the displayed history is always the server's.

B. Single-Player Mode

`NEW:AI` starts a game in which the human plays X and the server plays O. After each legal human move that does not end the game, the server chooses a random empty cell, plays it, and appends `:AI:<c>` to the reply, for example `OK:BOARD:..O.X....:AI:2`. The reply board already contains both moves, and the suffix is omitted when the human's move ends the game. If the computer's move ends the game, the reply is `WIN:O:<9>:AI:<c>` or `DRAW:<9>:AI:<c>`. Plain `NEW` remains human-versus-human with unchanged replies.

The computer picks the k-th empty cell, where k is a random number modulo the number of empty cells (Listing 3), which gives a uniform choice. The random number comes from a 32-bit xorshift generator that is stirred with the arrival time of every received command, so the sequence differs from run to run without extra hardware.

Listing 3. Computer opponent: random choice of an empty cell.

```
int game_ai_choose(uint32_t rnd)
{
    if (status != GS_PLAY) return -1;
    uint8_t empty = 9u - moves;
    if (empty == 0u) return -1;
    uint8_t k = (uint8_t)(rnd % empty);
    for (uint8_t i = 0; i < 9u; ++i)
        if (board[i] == '.') {
            if (k == 0u) return (int)i;
            --k;
        }
    return -1;
}
```

The chooser only suggests a cell. The server then plays it through the same `game_move()` checks as any human move, so the computer cannot make an illegal move or bypass win and draw detection. In a single-player game the human is always X, so `MOVE:O:<i>` is rejected with `NACK:NOTURN`.

C. Verification

The host-side C tests run the real server code on a PC. Exactly 300 complete single-player games, the tests check that the board changes only at the human's cell and the computer's cell, that the computer's cell was empty beforehand, that every non-terminal reply carries a computer move, and that `HIST` matches the moves actually played. Both X and O win across those games and the computer uses all nine cells over time. The chooser is also tested separately for a uniform spread over empty cells and for returning no cell once the game is over. Deliberately breaking the chooser to ignore occupancy made these tests fail, which confirms that they detect an illegal computer move.

PC-side checks. The pytest suite also exercises `NEW:AI` and `HIST` through the serial interface, and the GUI logic tests confirm that the client adopts the board, including the computer's reply, only from a valid server reply and replaces its history list from `HIST` after every accepted move.

VIII. Validation

A. Automated Test Harness

The suite is pytest, in three files. `test_link.py` covers framing and the checksum. `test_server.py` drives the serial interface with no human interaction. `test_client.py` covers the GUI logic. A fixture selects one of three targets: a Python fake server, the real firmware C logic running on the PC over pipes, or the real board through `SERIAL_PORT`. The same tests therefore run unchanged against all three. The assertions cover CONNECT and NEW, a legal move, the occupied-cell, out-of-range and out-of-turn NACKs, moves after game over, scripted win and draw sequences, resign, disconnect, bad checksums, oversized, garbage and partial frames and recovery afterwards, and, for the computer opponent, that the reply board changes only at the human and computer cells and that history matches the moves played. Rejected requests are checked to leave the board unchanged.

B. Test Results

Table V lists the results for the Python fake server and the real board; the same suite also runs against the firmware C logic on a PC. The board run skips the tests that need a

TABLE VI
Test results by target.

Target	Passed	Skipped
Python fake server	97	0
Real STM32 board	94	3

simulated button press; SW0 was pressed by hand and the LCD returned to IDLE (Fig. 7).

C. Demonstration

The figures show the working system on the real board. The demonstration video (see the Video section) also covers a two-player game, a game against the computer, a rejected move, the button reset and the test run.



Fig. 7. Board running, LCD showing IDLE (waiting for PC).

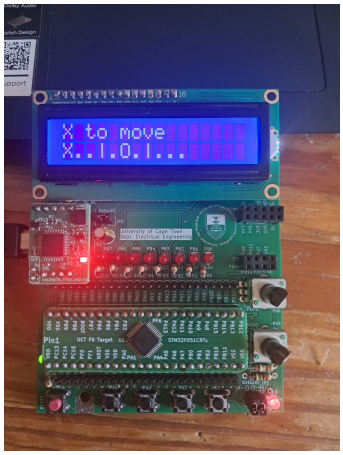


Fig. 8. Board mid-game: LCD shows the position and the LED shows the turn.



Fig. 9. Board after a game ends

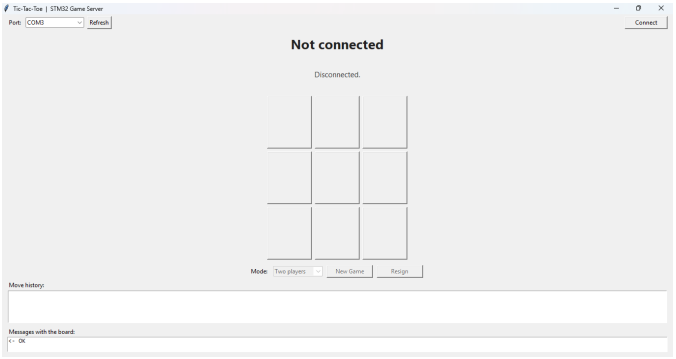


Fig. 10. GUI connected to the board, with the Mode selector.

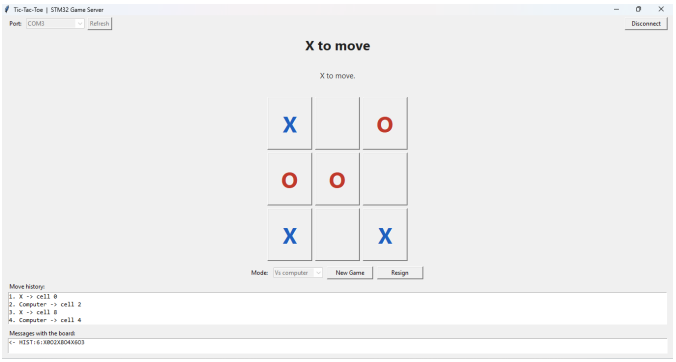


Fig. 11. GUI mid-game against the computer, with the move history list.

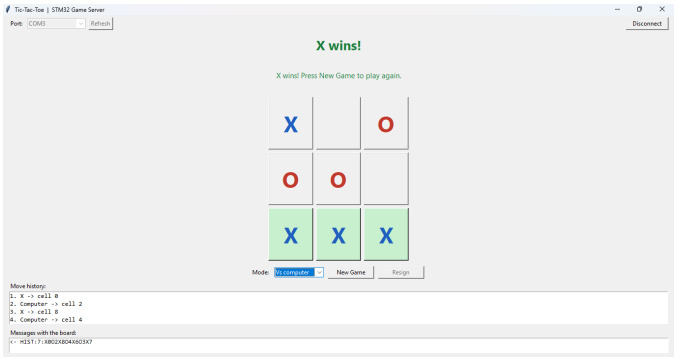


Fig. 12. GUI showing a win with the winning line highlighted.

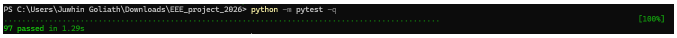


Fig. 13. Test suite passing against the Python fake server.



Fig. 14. Test suite passing against the real STM32 board.

IX. Conclusion

The system meets the brief: the STM32 is the sole authority, every illegal or malformed request is rejected with a specific NACK without changing state, and an automated harness verified this on the real board. Limitations: the computer opponent plays random legal moves rather than strategically, and after a reconnect the client cannot tell whether a game in progress is against the computer. Possible improvements are a minimax opponent and reporting the game mode in `STATE`.

Video

https://uctcloud-my.sharepoint.com/:f:/g/personal/gbrars001_myuct_ac_za/IgDBejbcy73dSpwEOb7HEPvaAWxst3mQ4_G_Wy7x9-m0Xso?e=uQ5Qcy

Code

https://github.com/GLTJUW001/EEE3095S-serial-game-server-project

References

[1] EEE3095/6S Practical 3 github for the STM32 base files
[2] Python Software Foundation, “tkinter — Python interface to Tcl/Tk,” https://docs.python.org/3/library/tkinter.html.
[3] pytest documentation, https://docs.pytest.org.
[4] pySerial documentation, https://pyserial.readthedocs.io.